1. Objectives:

1. To learn the fundamental concepts of string.
2. To understand how characters are stored and processed sequentially in memory.
3. To learn calculating string length, copying one string to another, and combining two strings into a single character array.
4. To apply the sliding window technique to determine whether a pattern string exists inside a larger string.

2. Introduction:

- **String:** Strings are sequences of characters a small set of elements. Unlike normal array, strings typically have smaller sets of items. String is terminated with a special null character ('\0'). This null character marks the end of the string and is essential for proper string manipulation. Unlike character arrays from C language (char[]), in C++ std::string handles memory management automatically and provides a wide range of built-in functions for ease of use. Strings can automatically grow and shrink as we add or remove characters. We can easily access characters, join strings, compare them, extract substrings, and search within strings using built-in functions. The number of characters in a string can be found using size() or length().

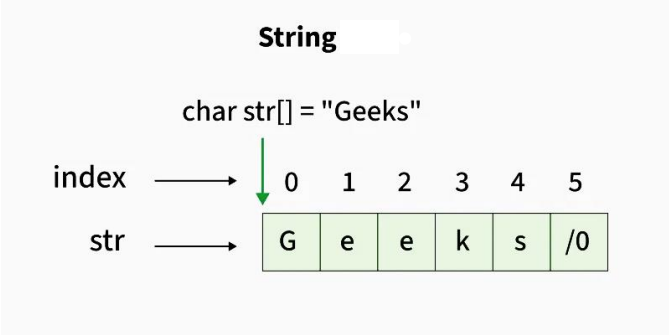

Fig:2.1: String

Copying is a process of transferring characters from one string (source) to another string or character array (destination) sequentially, until the null character ('\0') is reached. To copy a string there is a build in function called 'strcpy()' but in this lab we have use loops to copy a string manually. The process of joining two or more strings together to form a new string is called concatenation. The characters of the second string are appended to the end of the first string, and the resulting string is properly null terminated. In this lab we have joined 2 strings manually using loops.

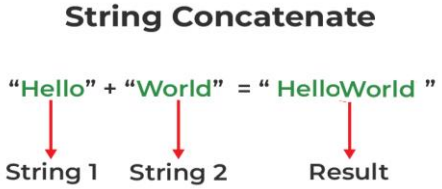

Fig:2.2: String Concatenation

- **Sliding Window:** Sliding Window Technique is a method used to solve problems that involve subarray, substring, or window. It is an efficient method used to process a fixed-size window over an array by moving it step by step and performing calculations on each window. Sliding window technique is used to check whether a smaller pattern string exists inside a larger text string. The window size is equal to the length of the pattern, and at each step, characters within the window are compared with the pattern. The window then shifts one position forward until a match is found or the entire string is checked.

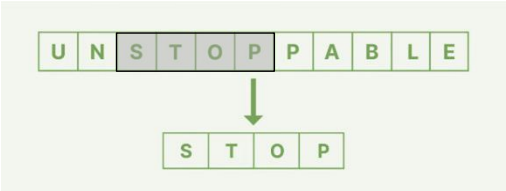

Fig:2.3: Finding string pattern

3. Procedure

Experiment 1: Basic String length calculation

Input:

- s – A string entered by the user
- i – Index variable for traversal
- c – Counter variable to calculate string length

Output:

- s – The entered string
- c – Length of the string

Algorithm:

1. Read string s
2. i ← 0
3. c ← 0
4. **while** s[i] ≠ '\0' **do**
5. c ← c + 1
6. i ← i + 1
7. **print** "Your String: ", s
8. **print** "String Length: ", c

Experiment 2: Copy String to another String

Input:

- p – Source string
- k – Character array where the string will be copied
- i – Index variable for traversal

Output:

- k – Copied string

Algorithm:

1. i ← 0
2. **while i < 3 do**
3. k[i] ← p[i]
4. i ← i + 1
5. k[i] ← '\0'
6. **print** "Copied: ", k

Experiment 3: Copy two Strings into one Array

Input:

- p – First given string
- q – Second given string
- k[] – Character array where both strings will be stored
- i – Index variable for first string
- j – Index variable for second string

Output:

- k – Combined string containing both p and q

Algorithm:

1. i ← 0
2. j ← 0
3. **while** p[i] ≠ '\0' **do**
4. k[i] ← p[i]
5. i ← i + 1
6. **while** q[j] ≠ '\0' **do**
7. k[i + j] ← q[i]
8. j ← j + 1
9. k[i + j] ← '\0'
10. **print** "Copied: ", k
11. **return** k

Experiment 4: Finding string pattern using Sliding Window

Input:

- T – Given text string
- P – Given string for pattern
- n1 – Length of text string

- n2 – Length of pattern string
- i – For text traversal
- j – For pattern comparison
- c – For matched characters

Output:

- i – Starting index where pattern is found

Algorithm:

1. n1 ← length(T)
2. n2 ← length(P)
3. found ← false
4. **for** i ← 0 to n1 − n2 **do**
5. c ← 0
6. **for** j ← 0 to n2 − 1 **do**
7. **if** P[j] == T[i + j] **then**
8. c ← c + 1
9. **else**
10. **break**
11. **if** c == n2 **then**
12. **print** "Pattern Found at position ", i
13. found ← true
14. **break**
15. **if** found == false **then**
15. **print** "Pattern Not Found"
16. **return**

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** Visual Studio Code 1.109

5. Experiment Implementation:

Experiment 1: Basic String length calculation

```
#include <bits/stdc++.h>
using namespace std;

int main(){

    string s;
    cout << "Enter String: ";
    cin >> s;
    cout << "Your String: " << s << endl;
```

```
    int i = 0;
    int c = 0;
    while (s[i] != '\0'){
        c = c + 1;
        i = i + 1;
    }
    cout << "String Length: " << c << endl;

    return 0;
}
```

Table 2.1: Experiment 1 output

Input	Output
Enter String: Rizen	Your String: Rizen String Length: 5

Experiment 2: Copy String to another String

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    string p = "ABCD";
    char k[5];
    int i = 0;

    while (i < 3){
        k[i] = p[i];
        i = i + 1;
    }
    k[i] = '\0';
    cout << "Copied: " << k << endl;

    return 0;
}
```

Table 2.2: Experiment 2 output

Copied: ABC

Experiment 3: Copy two Strings into one Array

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    string p = "ABCD";
    string q = "XYZ";
    char k[10];
    int i = 0;
    int j = 0;

    while (p[i] != '\0'){
        k[i] = p[i];
```

```
        i = i + 1;
    }

    while (q[j] != '\0'){
        k[i + j] = q[j];
        j = j + 1;
    }

    k[i + j] = '\0';
    cout << "Copied: " << k << endl;

    return 0;
}
```

Table 2.3: Experiment 3 output

Copied: ABCDXYZ

Experiment 4: Finding string pattern using Sliding Window

```
#include <iostream>
#include <string>
using namespace std;

int main(){
    string T = "BAUSTUNI";
    string P = "STU";

    int n1 = T.length();
    int n2 = P.length();

    bool found = false;
    for (int i = 0; i <= n1 - n2; i++){
        int c = 0;
        for (int j = 0; j < n2; j++){
            if (P[j] == T[i + j]){
                c = c + 1;
            }
            else{
                break;
            }
        }

        if (c == n2){
            cout << "Pattern Found at position " << i << endl;
            found = true;
            break;
        }
    }
    if (!found){
        cout << "Pattern Not Found" << endl;
    }

    return 0;
}
```

Table 2.4: Experiment 4 output

Pattern Found at position 3

6. Discussion & Conclusion:

In this lab, we have learned the fundamental concepts of strings such as how to calculate the length of a string, manually copy a string, string concatenation or copying two strings into one array manually, and pattern matching using the sliding window technique. We have learned how strings are stored, and how null termination works in character arrays. While implementing these programs, some challenges were faced in loop conditions, handling proper indexing, and ensuring correct null character placement during copy and concatenation operations. In the substring pattern matching part by understanding the logic of sliding window movement we had to be very careful. Logical mistakes, especially in nested loops, were more difficult to identify than simple syntax errors. Overall, we have successfully understood and applied the core concepts of string manipulation and pattern matching in C++.

7. Reference:

1. https://www.geeksforgeeks.org/dsa/string-data-structure/ accessed at 8.15 PM on 12th February 2026.
2. https://www.tutorialspoint.com/data_structures_algorithms/string_data_structure.htm accessed at 8.45 PM on 12th February 2026.